Decision Trees

BSAD 8310: Business Forecasting — Lecture 4

Department of Economics
University of Nebraska at Omaha

August 24, 2026

Lecture Outline

1. Why Go Beyond Linear Models?
2. How Decision Trees Work
3. Implementing Decision Trees in Python
4. Limitations and Preview: Ensembles

Why Go Beyond Linear Models?

Real-world relationships are often nonlinear, involve interactions, and shift with context.

The Limits of Linear Models

Three things a constant slope cannot represent:

- **Interactions** — the holiday-season advertising effect is not a constant slope
- **Threshold effects** — spending may be flat until unemployment drops below 5%, then jump
- **Asymmetry** — extreme values distort coefficients equally in both directions

Retail Sales Example

Monthly sales depend on last year's same-month sales (trend), whether it is December (seasonality), and the unemployment rate (conditions). The December effect is *larger* in a strong economy — an interaction a linear model misses.

Decision trees capture nonlinearity and interactions automatically, without your having to specify them in advance.

The Bias–Variance Tradeoff (James et al. 2023)

Bias–Variance Decomposition

$$\text{Expected MSE} = \underbrace{\text{Bias}^2}_{\text{systematic error}} + \underbrace{\text{Variance}}_{\text{sensitivity to data}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

High bias (underfitting):

- Model is too simple
- Misses real patterns in the data
- Bad on both training AND test sets
- Example: linear model on a curved relationship

High variance (overfitting):

- Model is too complex
- Memorizes training data noise
- Great on training, bad on test set
- Example: a very deep decision tree

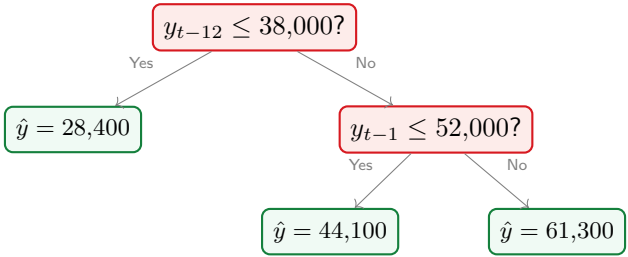
Goal: find the complexity that balances the two — which you can only see *out-of-sample*.

How Decision Trees Work

Partition the feature space into rectangles; predict the mean in each region.

What Is a Decision Tree?

A decision tree is a flowchart: ask yes/no questions about the features to arrive at a prediction.



Non-parametric · **nonlinear** (captures thresholds) · **interpretable** · **scale-invariant** · handles lags, rolling statistics, and dummies together. A single *deep* tree **overfits severely**. Ensembles are the fix, and are where the next two lectures go.

The CART Algorithm (Regression) (Breiman et al. 1984)

CART builds the tree greedily, top to bottom. At each node it searches over every feature j and threshold t for the split that most reduces within-region variance:

$$\min_{j,\,t}\left[\sum_{i:x_{ij}\le t}(y_i-\bar{y}_L)^2+\sum_{i:x_{ij}>t}(y_i-\bar{y}_R)^2\right]$$

Start with all the data at the root; split until nodes get too small or a depth cap is reached. The **prediction** at a leaf is simply the mean of the observations that land there. At every node the algorithm is really asking: *"what is the single yes/no question that best separates high-sales months from low-sales months?"* — then repeating it within each group. For classification, replace variance with Gini impurity or entropy (information gain).

Depth Controls the Bias–Variance Tradeoff

A **depth-1 stump** asks one question: high bias, low variance. A **full tree** with one observation per leaf memorizes the sample: zero bias, extreme variance.

Depth	Bias	Variance	Behavior
1 (stump)	High	Low	One question; underfits
5–10	Balanced	Balanced	Usually the useful range
Full tree	Zero	Extreme	Memorizes the training sample

Why trees are unstable: one changed observation can flip the split at the root, and every prediction below it changes with it. Tune `max_depth` and `min_samples_leaf` by `TimeSeriesSplit` cross-validation.

Implementing Decision Trees in Python

One class, a few hyperparameters, interpretable results.

Decision Tree Regression in Python

sklearn Code Template

```
from sklearn.tree import DecisionTreeRegressor
X = make_lag_features(df, lags=[1, 2, 3, 12]);  y = df['sales']
tree = DecisionTreeRegressor(max_depth=5, min_samples_leaf=10, random_state=42)
tree.fit(X_train, y_train);  y_hat = tree.predict(X_test)
```

Parameter	Effect
`max_depth`	Cap tree depth
`min_samples_leaf`	Minimum observations per leaf
`max_features`	Feature subsampling
`ccp_alpha`	Cost-complexity pruning

Cross-validate with `TimeSeriesSplit`, never `KFold` — **random folds leak the future into the training set**.

Feature Importance (Molnar 2022)

Impurity-Based Feature Importance

For each feature j, sum the total reduction in variance across all splits where j was used, weighted by the fraction of observations in each node. Normalize so the importances sum to 1.

Feature	Importance
Lag 12 (same month last year)	0.41
Lag 1 (previous month)	0.27
Lag 2	0.13
December dummy	0.10
Unemployment rate	0.09

This tells you which variables the tree found most useful for splitting — a fast way to screen predictors. But impurity importance is biased toward high-cardinality features. Permutation importance and SHAP give more reliable attributions; we return to this in Lecture 5.

Interpreting and Communicating Tree Predictions

A tree's main advantage is that you can explain *why* a specific prediction was made.

Explaining a Prediction to a Manager

"Our model forecasts $61,300 in sales for next month because:

1. Last year's same-month sales were above $38,000 (economy growing)
2. Last month's sales were above $52,000 (current momentum strong)
3. When both conditions hold, the average outcome in our training data was $61,300."

Produce the path with `sklearn.tree.plot_tree(tree)` or `export_text(tree, feature_names=[...])`. This is where business buy-in comes from — a manager can check that the tree is splitting on variables they consider sensible.

Limitations and Preview: Ensembles

A single tree is a weak learner. Combine hundreds to build a strong one.

When a Single Tree Falls Short

Strengths:

- ✓ Interpretable — can show the path
- ✓ No feature scaling needed
- ✓ Captures nonlinear relationships
- ✓ Handles missing values well
- ✓ Fast to train

Weaknesses:

- × High variance — small data changes give a very different tree
- × Flat predictions within each leaf
- × Cannot extrapolate beyond the training range
- × Sensitive to outliers in a leaf

The ensemble solution: average many trees trained on different bootstrap samples. Variance falls sharply while feature-importance interpretability survives — typically a 20–40% RMSE improvement over a single tree.

Lecture 4: Key Takeaways

1. Linear models cannot capture **interactions** or **threshold effects**. Decision trees can, without needing to specify these relationships in advance.
2. The **bias–variance tradeoff** means that a too-simple tree underfits (high bias) and a too-deep tree overfits (high variance). Tune with `max_depth` and `min_samples_leaf`.
3. **CART** greedily minimizes within-region variance at each split. Prediction = mean of observations in the leaf.
4. Decision trees are **interpretable**: you can trace the exact decision path that led to any prediction.
5. Single trees have **high variance** — they change dramatically when the data changes slightly. Ensembles (Random Forests, XGBoost) solve this.

Next: Random Forests — variance reduction through bootstrap aggregation and random feature selection.

References I

- Breiman, Leo et al. (1984). *Classification and Regression Trees*. Belmont, CA: Wadsworth.
- James, Gareth et al. (2023). *An Introduction to Statistical Learning with Applications in Python*. 1st. New York: Springer. URL: https://www.statlearning.com/.
- Molnar, Christoph (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 2nd. Open access. URL: https://christophm.github.io/interpretable-ml-book/.